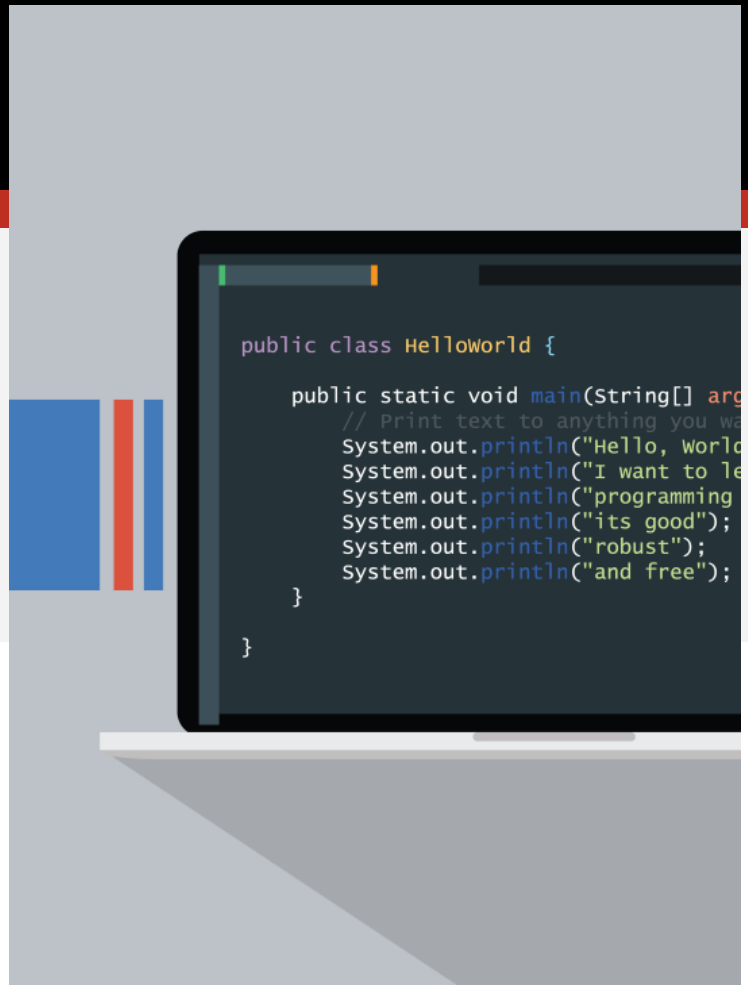
MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



Graph and Breadth-First Search

Table of Contents

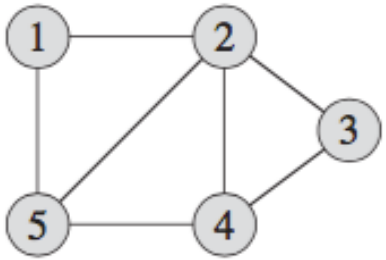
- **Graph and Breadth-First Search**
- Depth-First Search
- Topological Sort & Strongly Connected Component
- Single Source Shortest Path

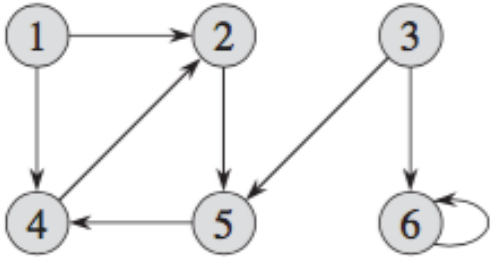
Graphs

- A **graph** is a fundamental data structure used to model relationships and connections.
- It's widely used in network design, navigation systems, social networks, etc.
- A graph is defined as

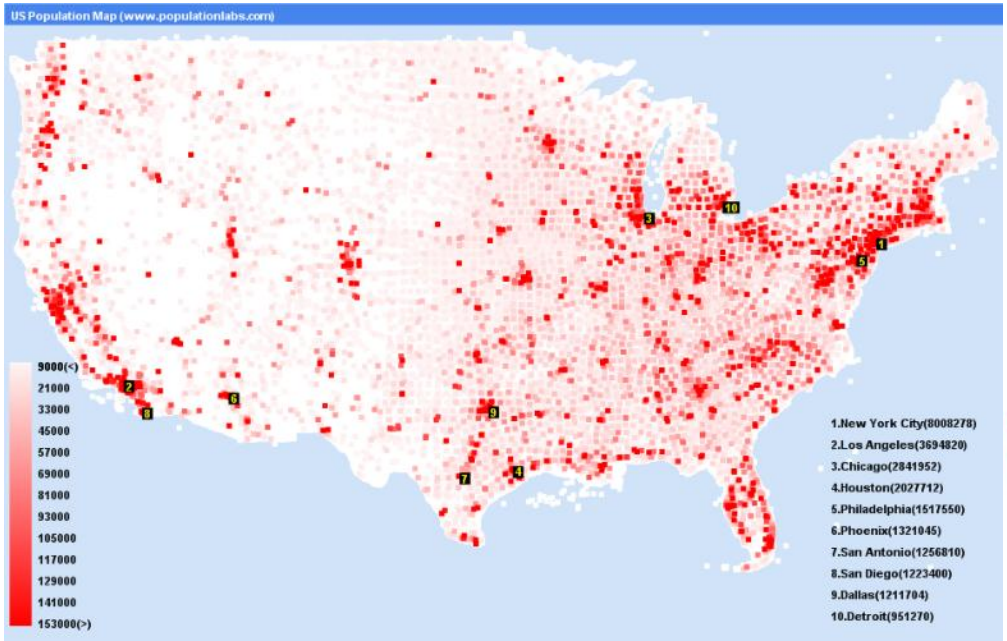
$$G = (V, E)$$

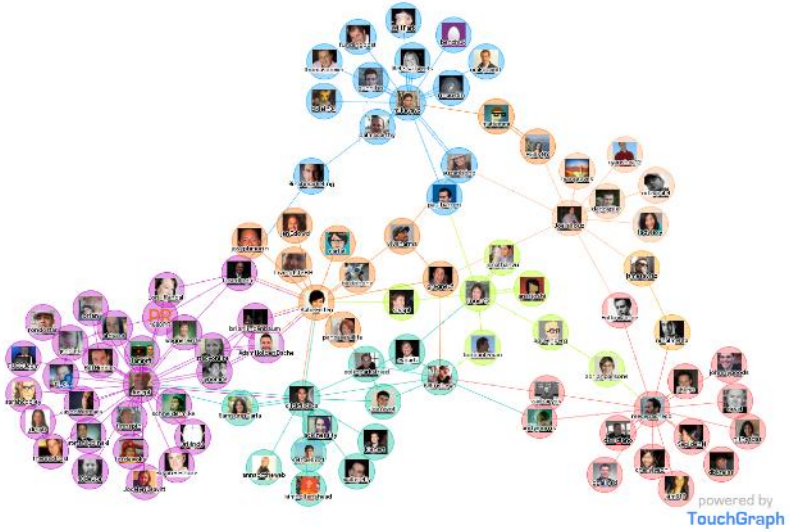
- Where:
  - V is a set of **vertices** (also called nodes)
  - E is a set of **edges** (connections between nodes)
- The edge can come with direction, or not
  - Undirected graph
  - Directed graph





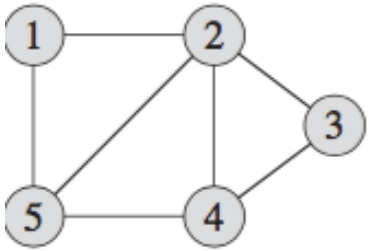
Examples of Graphs



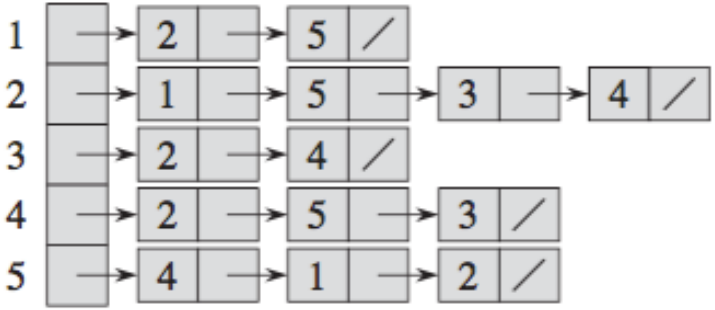


Graph Representations (1 of 2)

- The **adjacency-list representation** of a graph G=(V, E)
  - For each node v in V, create a linked list of nodes that have edge connection with v.



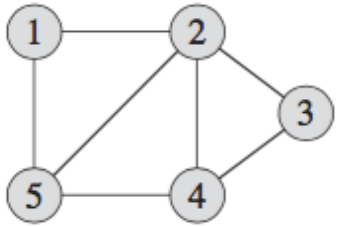
(a)



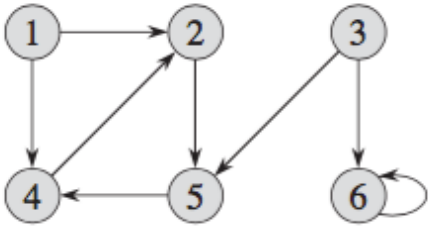
(b)

Graph Representations (2 of 2)

- The **adjacency-matrix representation** of a graph G=(V, E):
  - We create a matrix A whose dimension is |V|*|V|
  - We set $$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E , \\ 0 & \text{otherwise} . \end{cases}$$



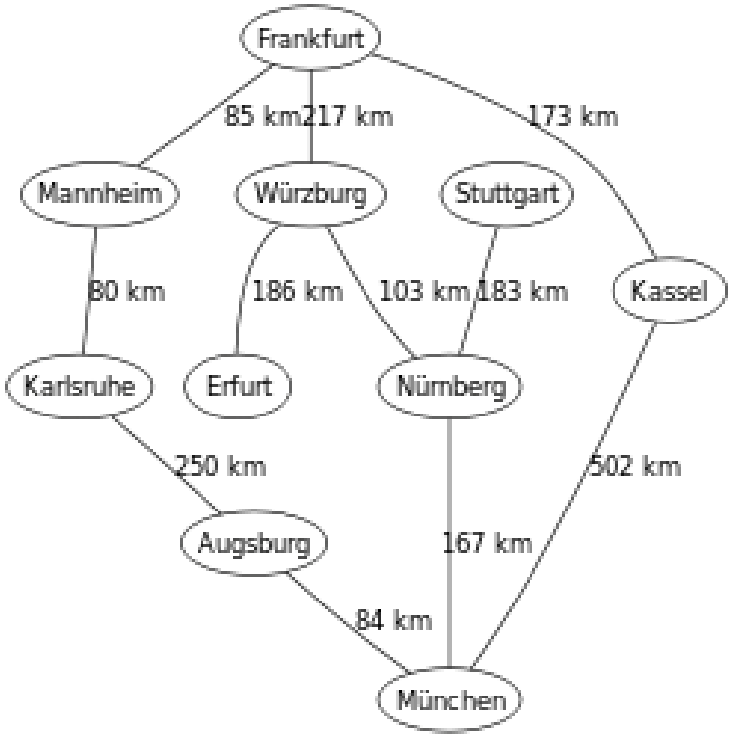
	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0



	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Breadth-First Search

- **Breadth-First Search (BFS)** is a fundamental **graph traversal algorithm** used to explore nodes level by level. It is widely applied in shortest path problems, tree traversals, and real-world network analysis.
- **Input:** A graph G=(V, E), a source node s
- **Output:** The set of destination nodes that are reachable from s, and their minimum distance to s.



Pseudocode

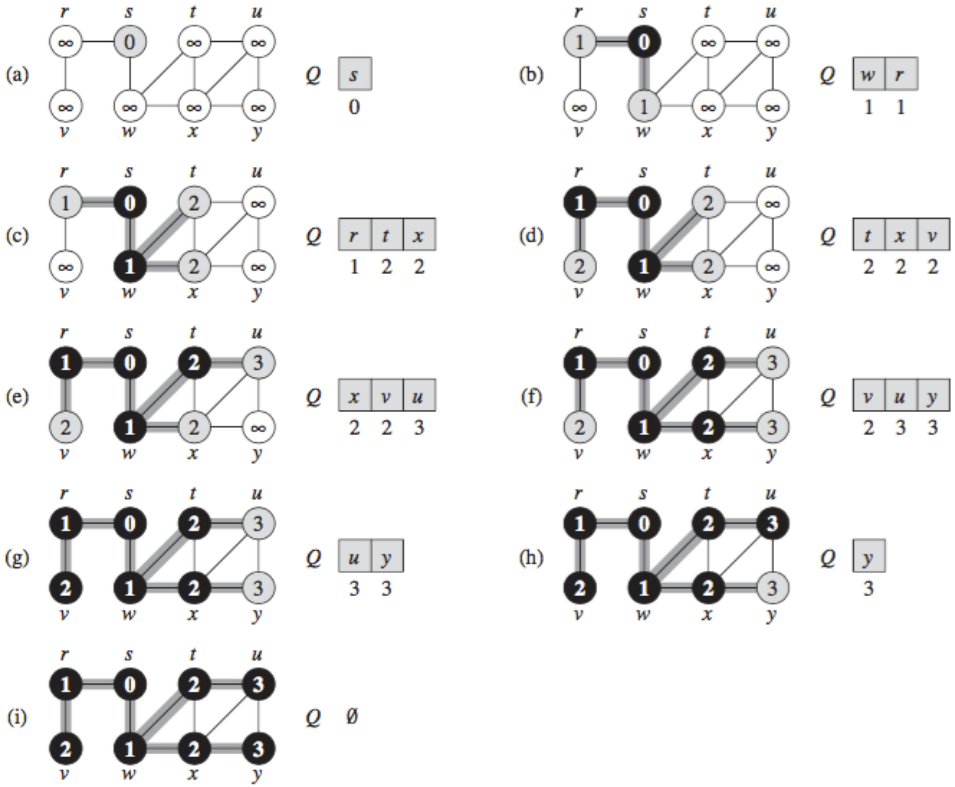
```
BFS(G, s)
for each vertex u ∈ G.V − {s}
    u.color = WHITE
    u.d = ∞
    u.π = NIL
s.color = GRAY
s.d = 0
s.π = NIL
Q = ∅
ENQUEUE(Q, s)
while Q ≠ ∅
    u = DEQUEUE(Q)
    for each v ∈ G.Adj[u]
        if v.color == WHITE
            v.color = GRAY
            v.d = u.d + 1
            v.π = u
            ENQUEUE(Q, v)
    u.color = BLACK
```

For each node v,

- v.d: the minimum distance to s
- v.π: the parent of v to reach s
- v.color:
  - WHITE: v has not been discovered yet
  - GRAY: v has been discovered, but not its neighbors
  - Black: v and its neighbors have been discovered. It will not be visited any more.

Example Walkthrough

```
BFS(G, s)
for each vertex u ∈ G.V − {s}
    u.color = WHITE
    u.d = ∞
    u.π = NIL
s.color = GRAY
s.d = 0
s.π = NIL
Q = ∅
ENQUEUE(Q, s)
while Q ≠ ∅
    u = DEQUEUE(Q)
    for each v ∈ G.Adj[u]
        if v.color == WHITE
            v.color = GRAY
            v.d = u.d + 1
            v.π = u
            ENQUEUE(Q, v)
    u.color = BLACK
```

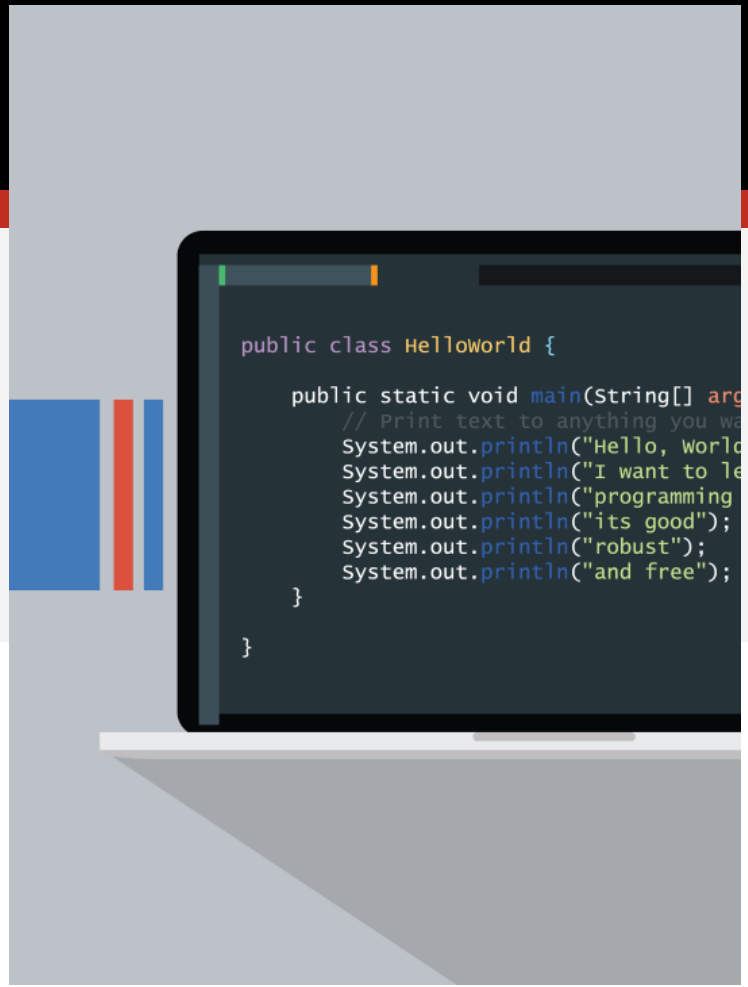


Complexity Analysis

- Time complexity: $O(|V|+|E|)$, because each node and edge is only visited once.

MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



Depth-First Search

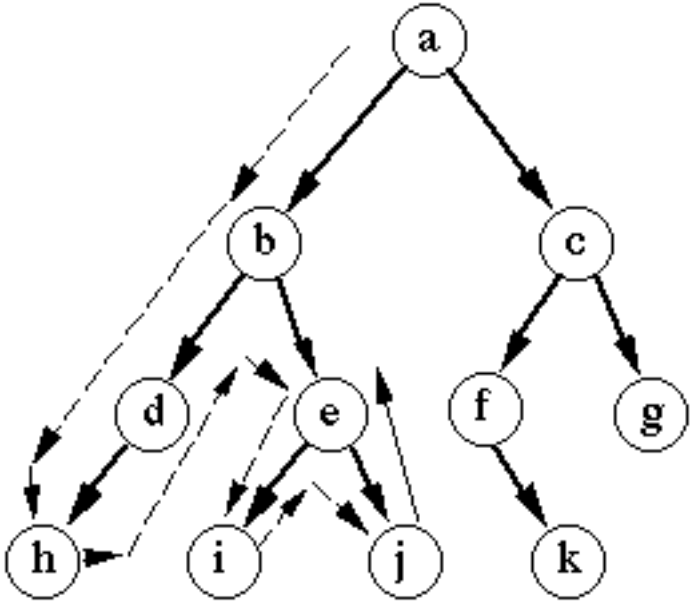
Table of Contents

- Graph and Breadth-First Search
- **Depth-First Search**
- Topological Sort & Strongly Connected Component
- Single Source Shortest Path

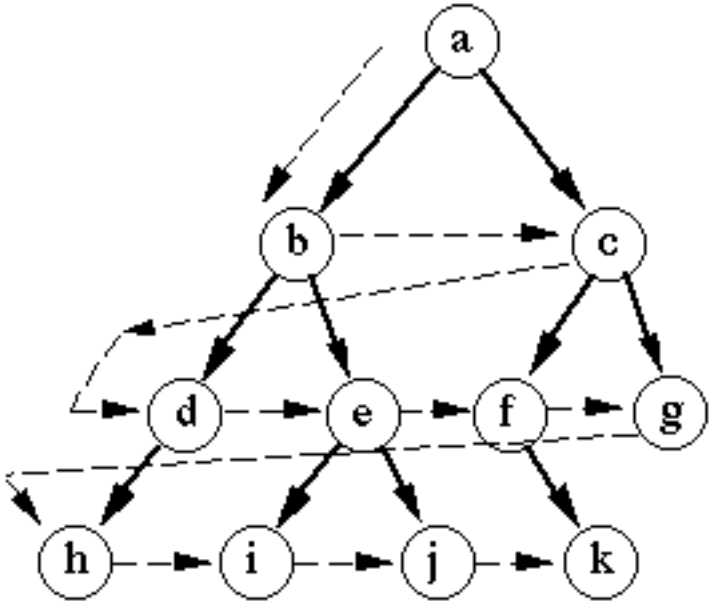
Depth-First Search

- **Depth-First Search (DFS)** is a fundamental graph traversal algorithm used to explore as far as possible along each branch before backtracking. It's ideal for solving pathfinding, connectivity, topological sorting, and cycle detection problems.
- In BFS, whenever we visit v for the first time, we will visit all of v's neighbors immediately after that.
- In DFS, whenever we visit v for the first time, for each neighbor u of v, we will keep going deep along u until we cannot proceed.

DFS vs. BFS



Depth-first search



Breadth-first search

Pseudocode

```
DFS(G)
for each vertex u ∈ G.V
    u.color = WHITE
    u.π = NIL
time = 0
for each vertex u ∈ G.V
    if u.color == WHITE
        DFS-VISIT(G, u)
```

```
DFS-VISIT(G, u)
time = time + 1
u.d = time
u.color = GRAY
for each v ∈ G.Adj[u]
    if v.color == WHITE
        v.π = u
        DFS-VISIT(G, v)
u.color = BLACK
time = time + 1
u.f = time
```

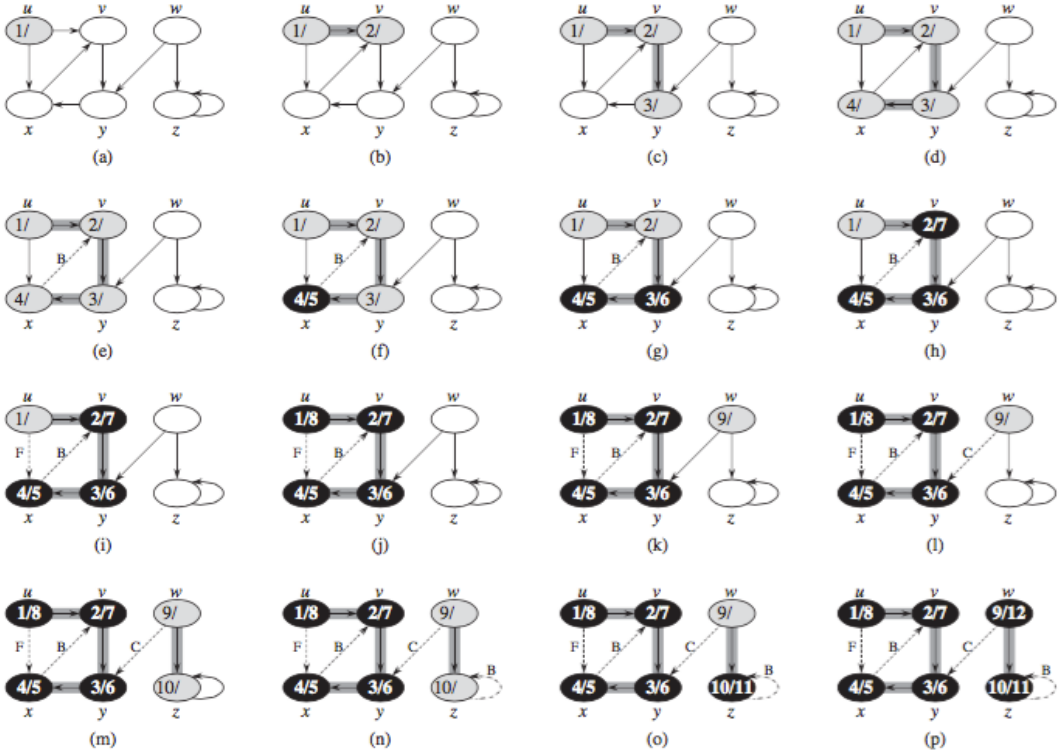
For each vertex u,

- u.d: the time that u turns from WHITE to GRAY
- u.f: the time that u turns from GRAY to BLACK
- u.π: the parent of u
- u.color:
  - WHITE: u has not been discovered yet
  - GRAY: u has been discovered, but not its neighbors
  - Black: u and its neighbors have been discovered. It will not be visited any more.

Example Walkthrough

```
DFS(G)
for each vertex u ∈ G.V
    u.color = WHITE
    u.π = NIL
time = 0
for each vertex u ∈ G.V
    if u.color == WHITE
        DFS-VISIT(G, u)
```

```
DFS-VISIT(G, u)
time = time + 1
u.d = time
u.color = GRAY
for each v ∈ G.Adj[u]
    if v.color == WHITE
        v.π = u
        DFS-VISIT(G, v)
u.color = BLACK
time = time + 1
u.f = time
```

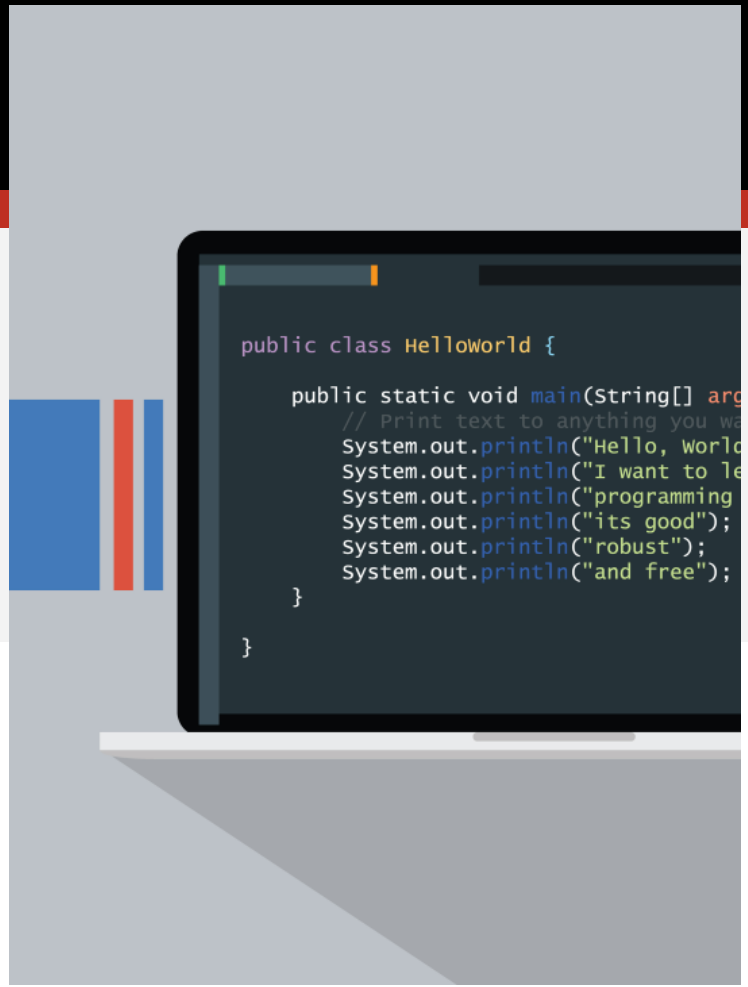


Complexity Analysis

- Time complexity: $O(|V| + |E|)$

MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



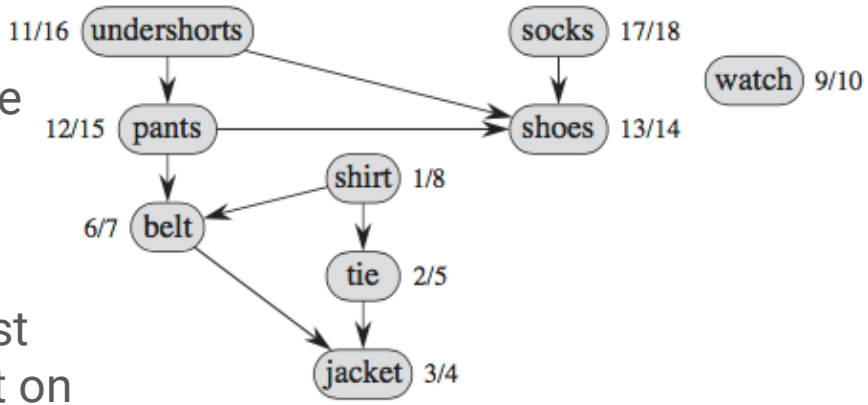
Topological Sort & Strongly Connected Component

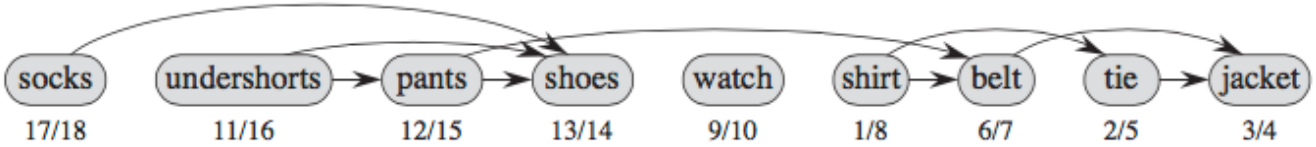
Table of Contents

- Graph and Breadth-First Search
- Depth-First Search
- **Topological Sort & Strongly Connected Component**
- Single Source Shortest Path

Topological Sort (1 of 2)

- **Topological Sort** is a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering.
- An application is that when someone gets dressed in the morning, there must be certain garments that should be put on before the others. For example, socks before shoes.





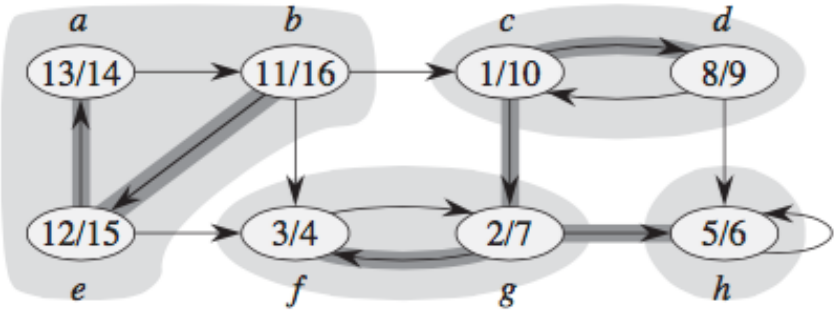
Topological Sort (2 of 2)

```
TOPOLOGICAL-SORT(G)
call DFS(G) to compute finishing times v.f for each vertex v
as each vertex is finished, insert it onto the front of a linked list
return the linked list of vertices
```

- Time complexity: $O(|V| + |E|)$

Strongly Connected Components (SCC)

- A strongly connected component of a directed graph $G = (V, E)$ is a maximal set of vertices $C \subseteq V$ such that for every pair of vertices u and v in C, both $u \rightarrow v$ and $v \rightarrow u$ exist in E.



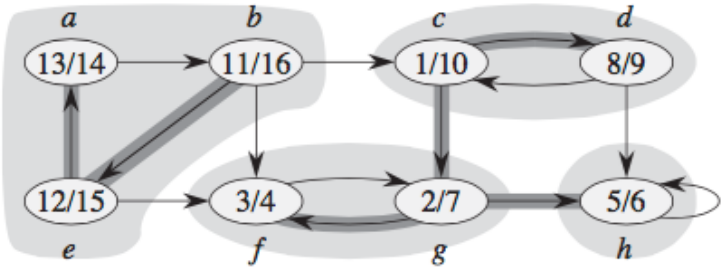
Pseudocode

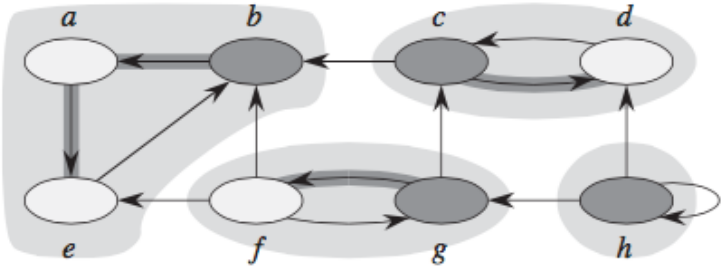
STRONGLY-CONNECTED-COMPONENTS(G)

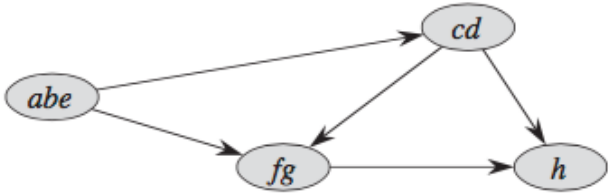
1 call DFS(G) to compute finishing times $u.f$ for each vertex u
2 compute G^{T}
3 call DFS(G^{T}), but in the main loop of DFS, consider the vertices
    in order of decreasing $u.f$ (as computed in line 1)
4 output the vertices of each tree in the depth-first forest formed in line 3 as a
    separate strongly connected component

- Time complexity: $O(|V| + |E|)$

Example Walkthrough

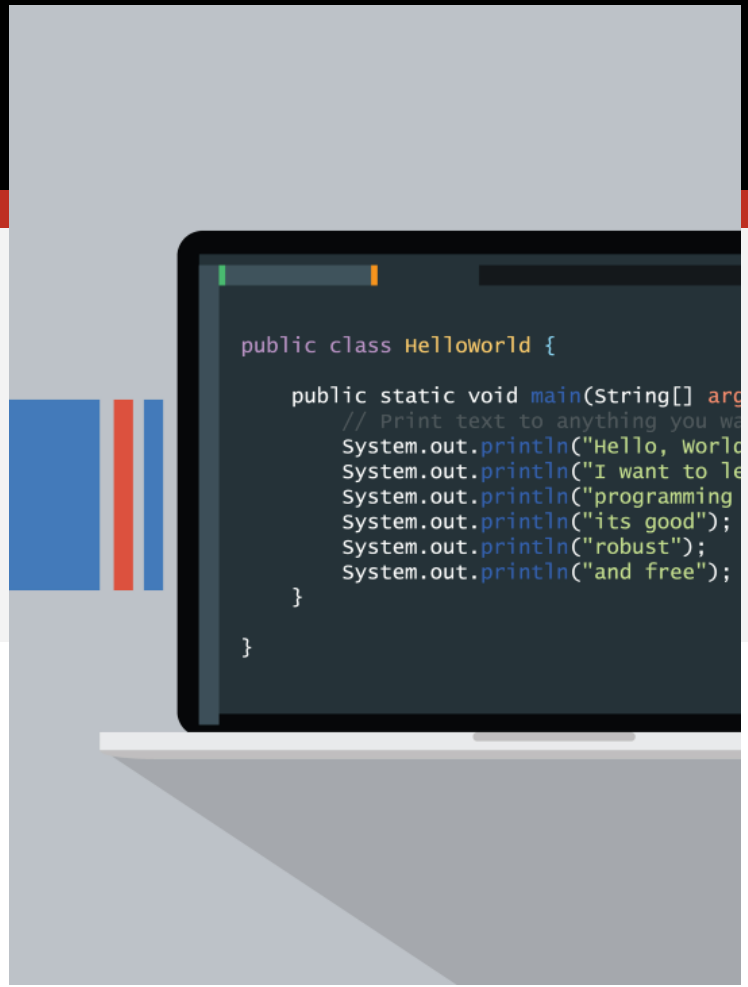






MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



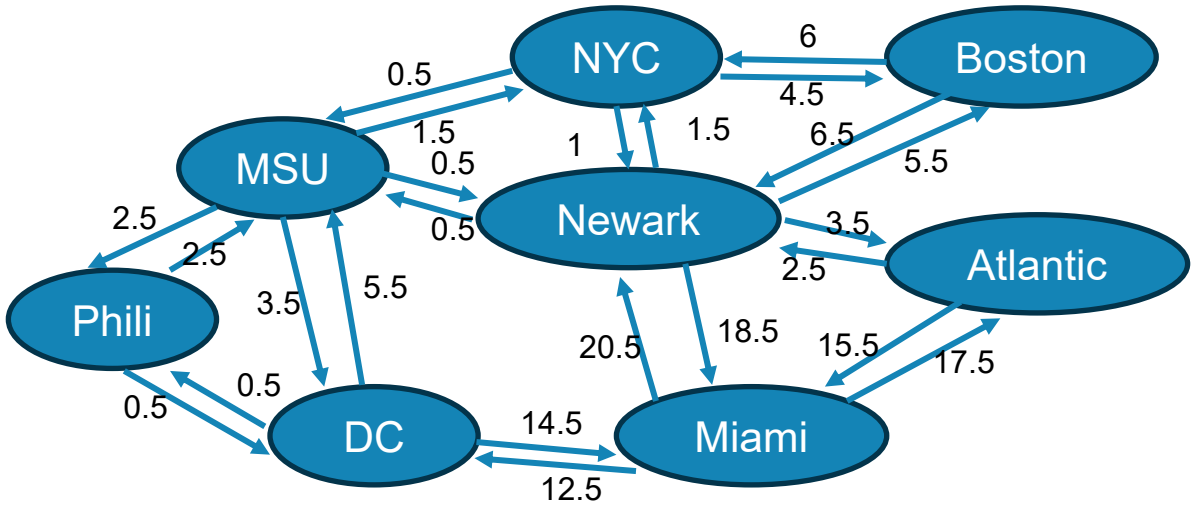
Single Source Shortest Path I

Table of Contents

- Graph and Breadth-First Search
- Depth-First Search
- Topological Sort & Strongly Connected Component
- **Single Source Shortest Path**

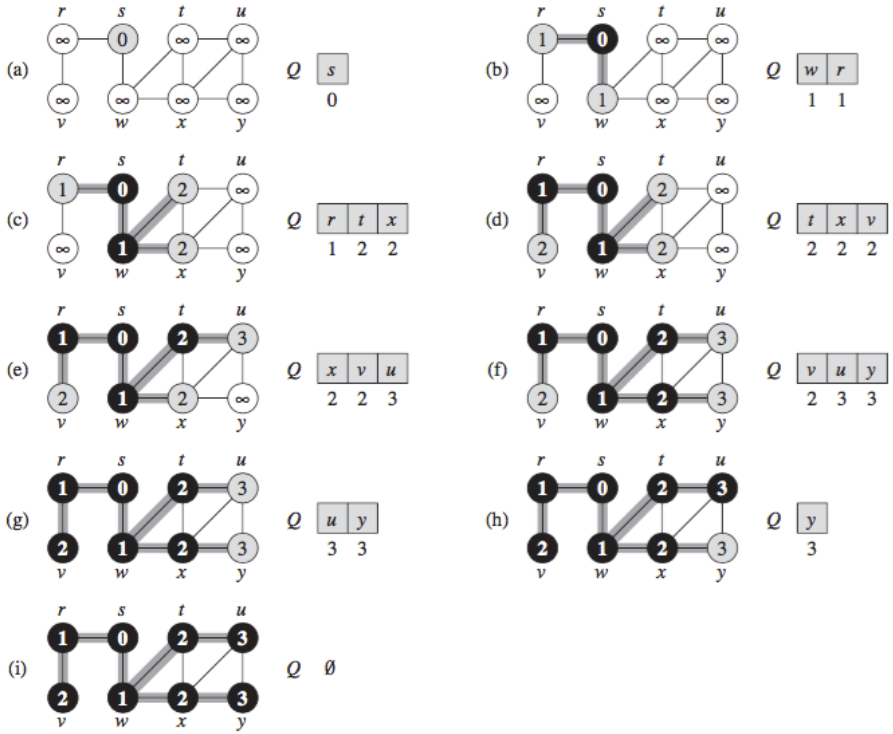
Single Source Shortest Path

- The Single Source Shortest Path (SSSP) problem focuses on finding the shortest path from one starting node (source) to all other nodes in a weighted graph.
  - Example: find the shortest driving time from NYC to all other cities.



Possible Solution: BFS (1 of 2)

```
BFS(G, s)
for each vertex u ∈ G.V − {s}
    u.color = WHITE
    u.d = ∞
    u.π = NIL
s.color = GRAY
s.d = 0
s.π = NIL
Q = ∅
ENQUEUE(Q, s)
while Q ≠ ∅
    u = DEQUEUE(Q)
    for each v ∈ G.Adj[u]
        if v.color == WHITE
            v.color = GRAY
            v.d = u.d + 1
            v.π = u
            ENQUEUE(Q, v)
    u.color = BLACK
```



Possible Solution: BFS (2 of 2)

- Limitation of BFS: it only considers if there is an edge or not, but does not consider the weight of the edges.
- So, we need a better solution to find the shortest path for a single source.

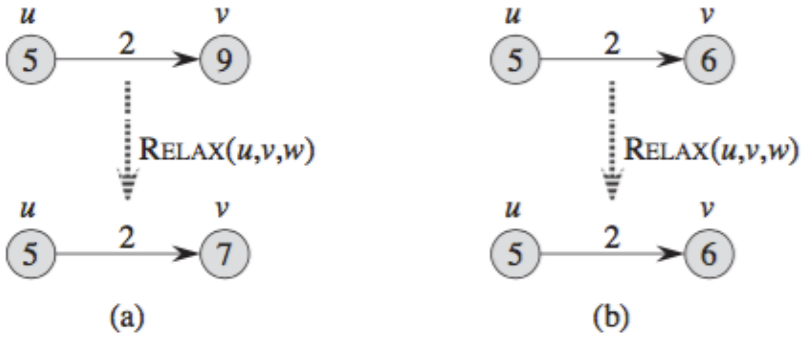
General Approaches

- **Initialization Step** common to all SSSP algorithms:

```
INITIALIZE-SINGLE-SOURCE(G, s)
for each vertex v ∈ G.V
    v.d = ∞
    v.π = NIL
s.d = 0
```

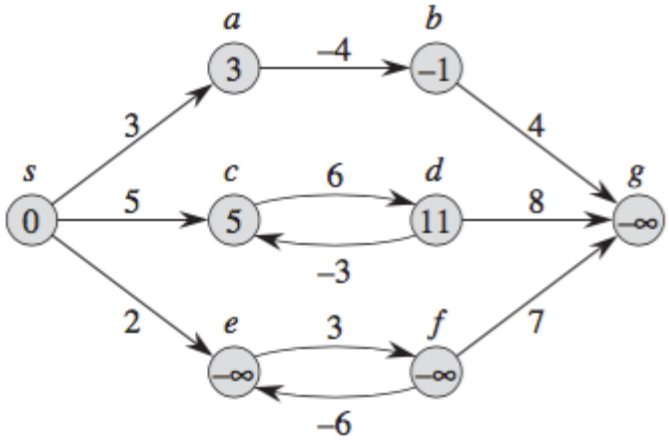
- **Relax edges**: try to update the distance to each neighbor if a shorter path is found.

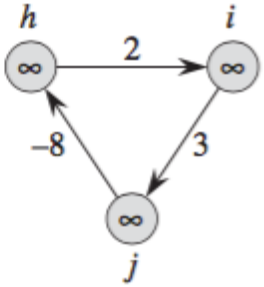
```
RELAX(u, v, w)
if v.d > u.d + w(u, v)
    v.d = u.d + w(u, v)
    v.π = u
```



Important Challenges

- Some factors we need to worry about:
  - 1. Negative edge weights
  - 2. Negative weight cycles





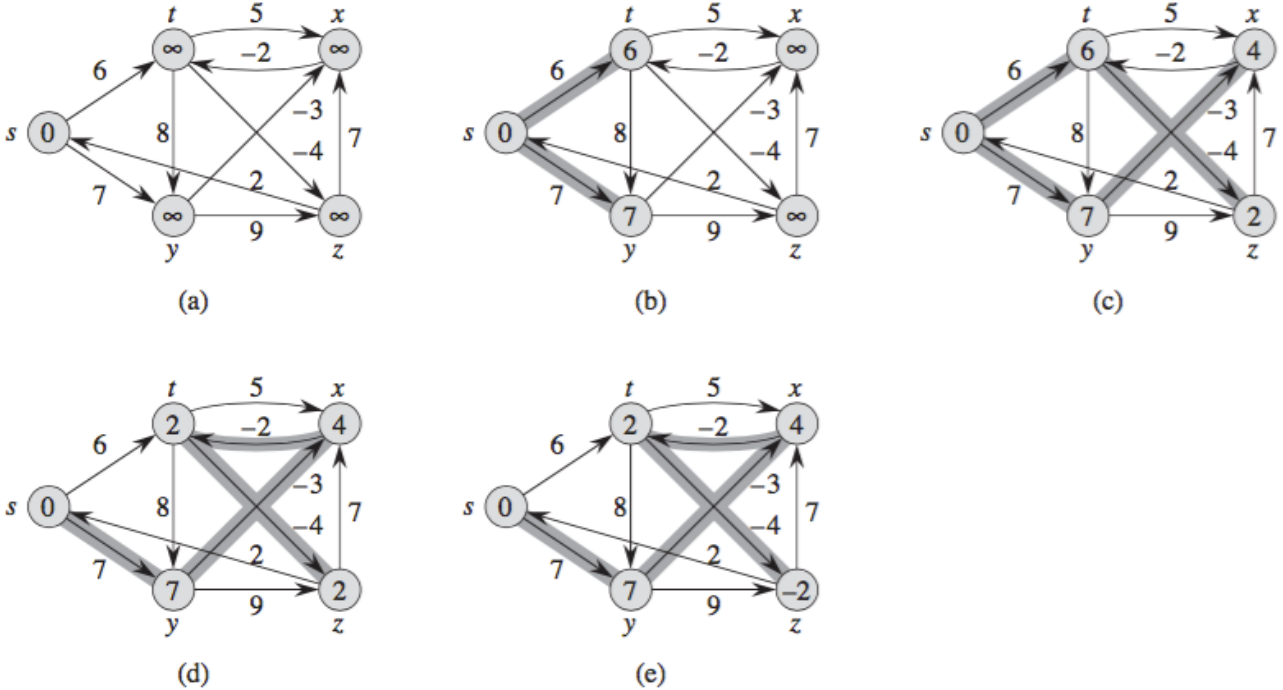
Bellman-Ford algorithm

- The first algorithm: **Bellman-Ford**
- **Condition**: allow negative-weight cycle.
  - Output false if the graph includes any negative-weight cycle.
- **Key idea**: a shortest path contains at most n nodes that are connected by n-1 edges, so for each destination, relax at most n-1 times.

```
BELLMAN-FORD(G, w, s)
INITIALIZE-SINGLE-SOURCE(G, s)
for i = 1 to |G.V| - 1
    for each edge (u, v) ∈ G.E
        RELAX(u, v, w)
for each edge (u, v) ∈ G.E
    if v.d > u.d + w(u, v)
        return FALSE
return TRUE
```

Example Walkthrough

- Time complexity: $O(VE)$



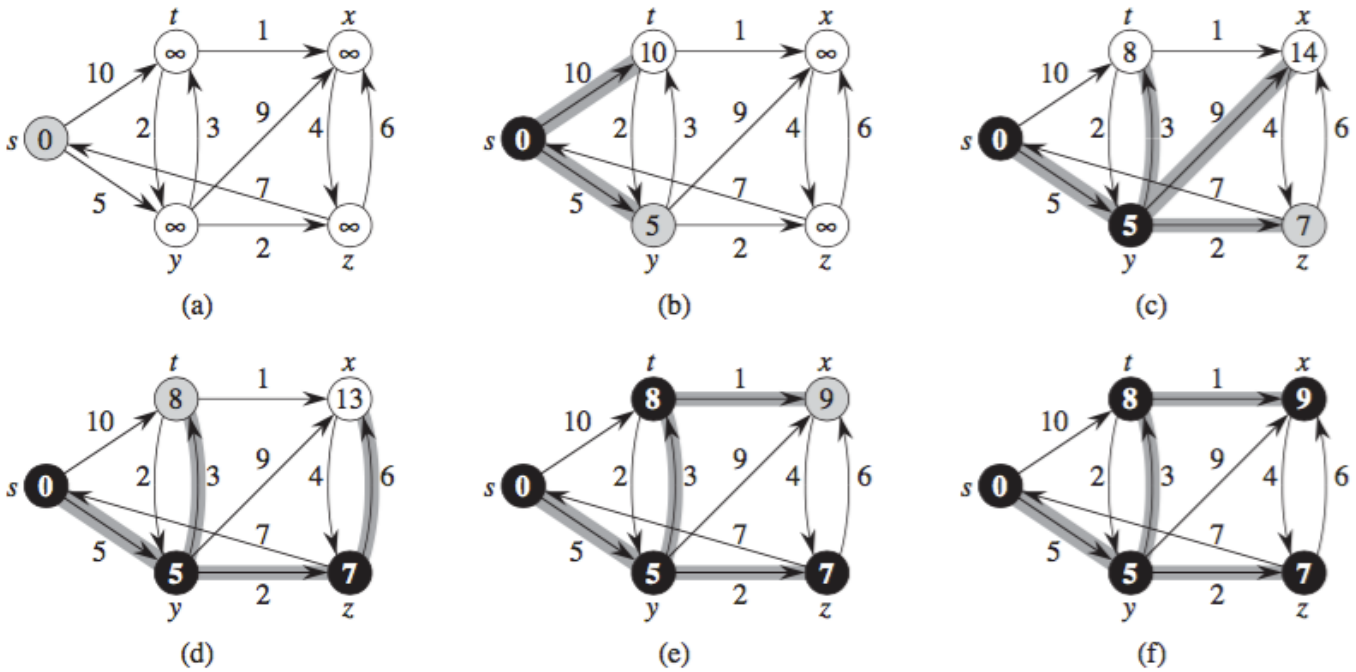
Dijkstra's Algorithm

- The second algorithm: **Dijkstra**
- **Condition**: non-negative edge weight
- **Key idea**: Maintain a set S of vertices whose final shortest-path weights have been determined. In each iteration, add one node to S.

```
DIJKSTRA(G, w, s)
INITIALIZE-SINGLE-SOURCE(G, s)
S = ∅
Q = G.V
while Q ≠ ∅
    u = EXTRACT-MIN(Q)
    S = S ∪ {u}
    for each vertex v ∈ G.Adj[u]
        RELAX(u, v, w)
```

Example Walkthrough

- Time complexity: $O(VlogV + E)$



Reading

- Chapter 22.1 - 22.5
- Chapter 24.1 – 24.3

MONTCLAIR
STATE UNIVERSITY